

Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures

Chenggang Zhao
DeepSeek-AI
Beijing, China
chenggangz@deepseek.com

Damai Dai
DeepSeek-AI
Beijing, China
damai.dai@deepseek.com

Liyue Zhang*

DeepSeek-AI
Beijing, China
ly.zhang@deepseek.com

Shirong Ma
DeepSeek-AI
Beijing, China
mashirong.2000@deepseek.com

Yuqing Wang*

DeepSeek-AI
Beijing, China
wangyq@deepseek.com

Chengqi Deng
DeepSeek-AI
Beijing, China
cq.deng@deepseek.com

Huazuo Gao
DeepSeek-AI
Beijing, China
gaohuazuo@deepseek.com

Panpan Huang
DeepSeek-AI
Beijing, China
pp.huang@deepseek.com

Wenfeng Liang
DeepSeek-AI
Beijing, China
wenfeng.liang@deepseek.com

Yuxuan Liu
DeepSeek-AI
Beijing, China
liuyuxuan@deepseek.com

Chong Ruan
DeepSeek-AI
Beijing, China
chong.ruan@deepseek.com

Jiashi Li
DeepSeek-AI
Beijing, China
js.li@deepseek.com

Shangyan Zhou
DeepSeek-AI
Beijing, China
sy.zhou@deepseek.com

Ying He
DeepSeek-AI
Beijing, China
ying.he@deepseek.com

Y.X. Wei
DeepSeek-AI
Beijing, China
weiyx@deepseek.com

Abstract

The rapid scaling of large language models (LLMs) has unveiled critical limitations in current hardware architectures, including constraints in memory capacity, computational efficiency, and interconnection bandwidth. DeepSeek-V3, trained on 2,048 NVIDIA H800 GPUs, demonstrates how hardware-aware model co-design can effectively address these challenges, enabling cost-efficient training and inference at scale. This paper presents an in-depth analysis of the DeepSeek-V3/R1 model architecture and its AI infrastructure, highlighting key innovations such as Multi-head Latent Attention (MLA) for enhanced memory efficiency, Mixture of Experts (MoE) architectures for optimized computation-communication trade-offs, FP8 mixed-precision training to unlock the full potential of hardware capabilities, and a Multi-Plane Network Topology to minimize

cluster-level network overhead. Building on the hardware bottlenecks encountered during DeepSeek-V3's development, we engage in a broader discussion with academic and industry peers on potential future hardware directions, including precise low-precision computation units, scale-up and scale-out convergence, and innovations in low-latency communication fabrics. These insights underscore the critical role of hardware and model co-design in meeting the escalating demands of AI workloads, offering a practical blueprint for innovation in next-generation AI systems.

CCS Concepts

• Computer systems organization → Architectures.

Keywords

Large Language Model, Mixture-of-Experts, Deep Learning, FP8 Mixed-Precision Training, Multi-Plane Network, Co-Design

*Yuqing Wang and Liyue Zhang are the corresponding authors of this paper. Authors are listed in alphabetical order of their first names.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1261-6/25/06
<https://doi.org/10.1145/3695053.3731412>

ACM Reference Format:

Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, and Y.X. Wei. 2025. Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3695053.3731412>

1 Introduction

1.1 Background

Large Language Models (LLMs) have undergone rapid evolution in recent years, driven by iterative advancements in model design, computational power, and data availability. In 2024, groundbreaking models such as GPT4o [59], LLaMa-3 [3], Claude 3.5 Sonnet [8], Grok-2 [73], Qwen2.5 [75], Gemini-2 [37] and our DeepSeek-V3 [26] have showcased remarkable progress, further narrowing the gap towards Artificial General Intelligence (AGI). As the Scaling Laws [45] shows, increasing model size, training data, and computational resources leads to substantial improvements in model performance, underscoring the pivotal role of scaling in advancing AI capabilities. Collectively, these developments have ushered in an era where scaling model size and computational power is seen as the key to unlocking higher levels of intelligence.

Recent developments, reasoning models such as OpenAI's o1/o3 series models [60, 61], DeepSeek-R1 [28], Claude-3.7 Sonnet [9], Gemini 2.5 Pro [38], Seed1.5-Thinking [68] and Qwen3 [71] have demonstrated not only the benefits conferred by large-scale architectures, but also the necessity of improving inference efficiency, particularly in handling longer contexts and achieving greater reasoning depth. These advancements underscore the need for faster and more efficient inference, consequently placing ever-increasing demands on computational resources.

To meet these challenges, industry leaders such as Alibaba, ByteDance, Google, xAI and Meta have deployed colossal training clusters [33, 42, 43, 56, 62, 74], featuring tens or even hundreds of thousands of GPUs or TPUs. While such massive infrastructures have enabled the development of state-of-the-art models, their exorbitant costs present significant barriers for smaller research teams and organizations. Despite these barriers, open-source startups such as DeepSeek [23–26, 28] and Mistral [41, 55] are also striving to develop state-of-the-art models. Among them, DeepSeek has especially demonstrated that effective software-hardware co-design can enable cost-efficient training of large models, leveling the playing field for smaller teams.

Building on this tradition, DeepSeek-V3 [26] represents a new milestone in cost-effective training. By leveraging just 2,048 NVIDIA H800 GPUs, DeepSeek-V3 achieves state-of-the-art performance. This achievement aligns with the commitment to advance AI through practical and scalable solutions, as previously demonstrated in the cost-effective architecture of Fire-Flyer AI-HPC [7]. The practices and insights derived from DeepSeek-V3 demonstrate how existing hardware resources can be harnessed to their fullest potential, offering valuable lessons for the broader AI and HPC communities.

1.2 Objectives

This paper does not aim to reiterate the detailed architectural and algorithmic specifics of DeepSeek-V3, which are extensively documented in its technical report [26]. Instead, it adopts a dual perspective—spanning hardware architecture and model design—to explore the intricate interplay between them in achieving cost-efficient large-scale training and inference. By examining this synergy, we aim to provide actionable insights for scaling LLMs efficiently without sacrificing performance or accessibility.

Specifically, the paper focuses on:

- **Hardware-Driven Model Design:** Analyze how hardware features, such as FP8 low-precision computation and scale-up/scale-out network properties, informed the architectural choices in DeepSeek-V3.
- **Mutual Dependencies Between Hardware and Models:** Investigate how hardware capabilities shape model innovation and how the evolving demands of LLMs drive the need for next-generation hardware.
- **Future Directions for Hardware Development:** Derive actionable insights from DeepSeek-V3 to guide the co-design of future hardware and model architectures, paving the way for scalable, cost-efficient AI systems.

1.3 Structure of this Paper

The remainder of this paper is organized as follows. Section 2 explores the design principles underpinning DeepSeek-V3 model architecture, highlighting key innovations such as Multi-head Latent Attention, Mixture-of-Experts optimizations and Multi-Token Prediction Module. Section 3 illustrates how our model architecture pursues low-precision computation and communication. Section 4 includes scale-up interconnection optimizations, discusses scale-up/scale-out convergence, and explores how hardware features influence parallelism and expert selection strategies. Section 5 focuses on scale-out network optimizations, including multi-plane network co-designs and low-latency interconnects. Besides current limitations and future suggestions mentioned in Section 3~5, Section 6 elaborates on more critical insights from DeepSeek-V3, and identifies directions for future hardware and model co-design.

2 Design Principles for DeepSeek Models

The development of **DeepSeek-V3** exemplifies a hardware-aware approach to scaling LLMs, where each design decision was carefully aligned with hardware constraints to optimize performance and cost efficiency.

As shown in Figure 1, DeepSeek-V3 employs the **DeepSeek-MoE** [27] and **Multi-head Latent Attention (MLA)** [25] architectures that have been proven effective in DeepSeek-V2 [25]. DeepSeek-MoE unlocks the potential of MoE architecture, while MLA drastically reduces memory consumption by compressing Key-Value (KV) caches. In addition, **DeepSeek-V3** incorporates **FP8 mixed-precision training**, significantly lowering computational costs and making large-scale training more practical without compromising model quality. To improve the inference speed, DeepSeek-V3 integrates speculative decoding based on its **Multi-Token Prediction Module**, which significantly increases the generation speed. Beyond model architecture, we also explored cost-efficient AI infrastructure by deploying a **Multi-Plane** two-layer Fat-Tree network to replace a traditional three-layer Fat-Tree topology, reducing cluster networking costs.

These innovations aim to address three core challenges in scaling LLMs—**memory efficiency**, **cost-effectiveness**, and **inference speed**—which are explored in detail in the following subsections.

2.1 Memory Efficiency

LLMs generally require significant memory resources, with memory demands increasing by more than 1000% per year. In contrast, the

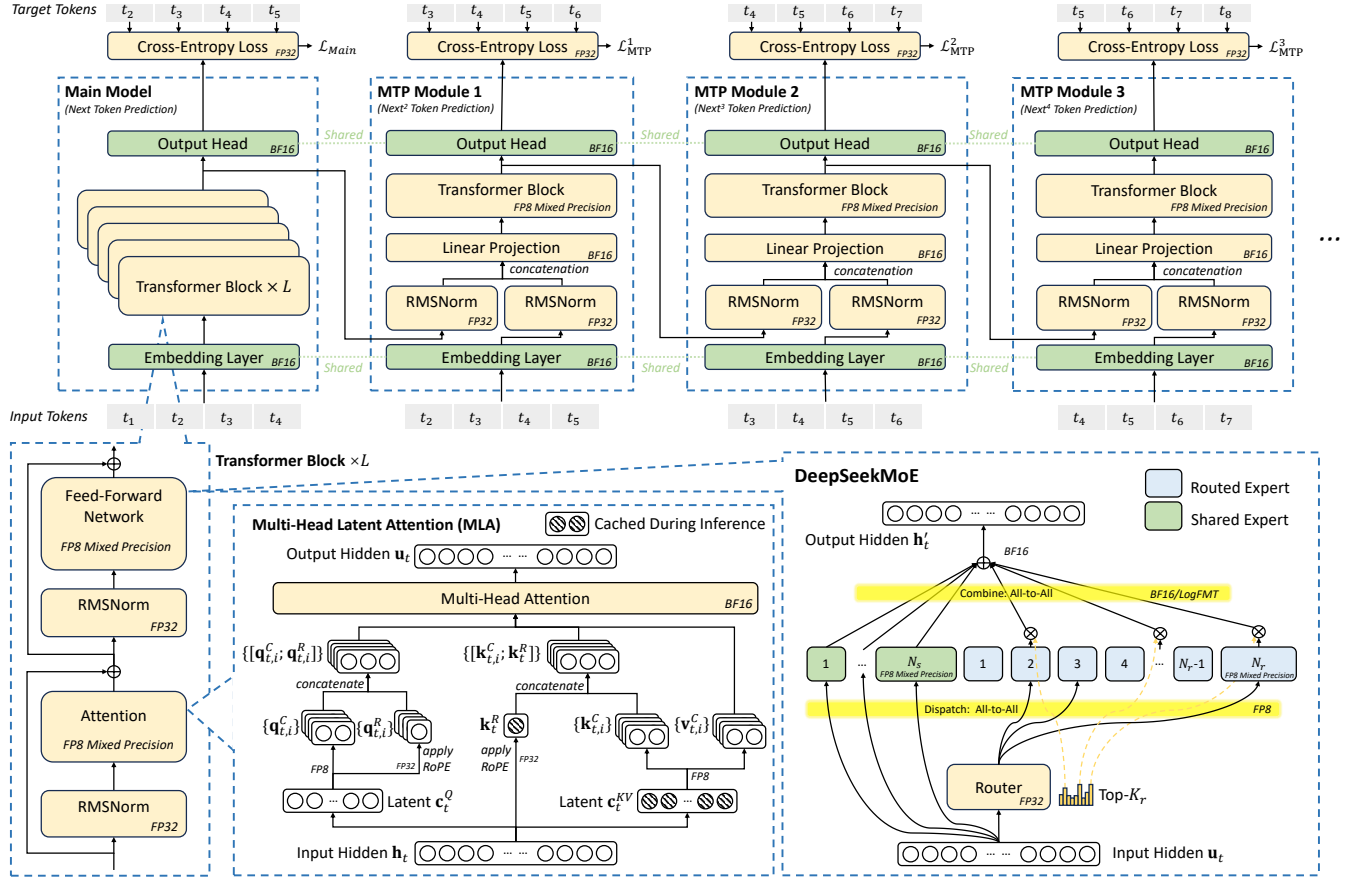


Figure 1: Basic architecture of DeepSeek-V3. Built upon DeepSeek-V2’s MLA and DeepSeekMoE, a Multi-Token Prediction Module and FP8 mixed-precision training are introduced to enhance inference and training efficiency. The figure indicates the precision used for computations in different parts of the architecture. All components take inputs and outputs in BF16.

growth rate of high-speed memory (e.g., HBM) capacity is much slower, typically less than 50% per year [35]. While multi-node parallelism is a viable solution to address memory limitations, optimizing memory usage at the source remains a crucial and effective strategy.

2.1.1 Low-Precision Models. Compared to models that utilize BF16 for weights, FP8 significantly reduces memory consumption by half, effectively alleviating the AI memory wall challenge. A detailed discussion of low-precision techniques is provided in Section 3 Low-Precision Driven Design.

2.1.2 Reducing KV Cache with MLA. For LLM inference, user requests often involve multi-turn conversations. To handle these efficiently, the context from previous requests is cached in what is commonly referred to as the *KV cache*. KV cache addresses this challenge by caching the **Key** and **Value** vectors of previously processed tokens, eliminating the need to recompute them for subsequent tokens. During each inference step, the model only computes the Key and Value vectors for the current token and performs attention computation by combining them with the cached Key-Value pairs from the history. This incremental computation reduces the complexity of generating each token to $O(N)$, making it efficient when processing long sequences or multi-turn inputs. However, it introduces

a memory-bound bottleneck because the computation shifts from GEMM to GEMV, which has a much lower compute-to-memory ratio. With modern hardware offering hundreds of TFLOPS, GEMV quickly becomes limited by memory bandwidth, making memory access the primary bottleneck.

To address this bottleneck, we employ **Multi-head Latent Attention (MLA)** [25] that compresses the KV representations of all attention heads into a smaller latent vector using a projection matrix, which is jointly trained with the model. During inference, only the latent vector needs to be cached, significantly reducing memory consumption compared to storing the KV cache for all attention heads.

In addition to MLA, several other approaches have been proposed to reduce the size of the KV cache. These methods are highly valuable and provide significant inspiration for advancements in memory-efficient attention mechanisms:

- **Shared KV (Grouped-Query Attention, GQA; Multi-Query Attention, MQA):** Instead of maintaining separate KV pairs for each attention head, multiple heads share a single set of KV pairs, significantly compressing KV storage. Representative methods include GQA [5] and MQA [70].

Table 1: KV cache size comparison (BF16 precision): DeepSeek-V3 (MLA) largely reduces KV cache size compared to other models using GQA.

Model	KV Cache Per Token	Multiplier
DeepSeek-V3 (MLA)	70.272 KB	1x
Qwen-2.5 72B (GQA)	327.680 KB	4.66x
LLaMA-3.1 405B (GQA)	516.096 KB	7.28x

- **Windowed KV:** For long sequences, only a sliding window of KV pairs is retained in the cache, discarding results outside the window. While this reduces storage, it compromises long-context reasoning. Representative methods include Longformer [11] and related architectures.
- **Quantized Compression:** KV pairs are stored using low-bit representations [40, 44, 52], further reducing memory usage. Quantization achieves significant compression with minimal impact on model performance.

Table 1 compares the KV cache memory usage per token among DeepSeek-V3, Qwen-2.5 72B [75], and LLaMA-3.1 405B [4]. By adopting MLA, DeepSeek-V3 achieves a significant reduction in KV cache size, requiring only 70 KB per token, substantially less than LLaMA-3.1 405B’s 516 KB and Qwen-2.5 72B’s 327 KB. This reduction highlights the efficiency of MLA in compressing KV representations compared to GQA-based methods. The ability to achieve such a significant reduction in memory consumption makes DeepSeek-V3 particularly well-suited for scenarios involving long-context processing and resource-constrained environments, enabling more scalable and cost-effective inference.

2.1.3 Future Directions and Perspectives on Resource-Efficient Techniques. While reducing the size of the KV cache is a promising method for improving memory efficiency, the quadratic complexity inherent in Transformer-based autoregressive decoding remains a formidable challenge, especially for extremely long contexts. Recent research efforts, such as Mamba-2 [21] and Lightning Attention[63], investigate linear-time alternatives that offer new possibilities for balancing computational cost and model performance. In addition, approaches such as sparse attention [76], which seek to compress and sparsely activate attention keys and values, represent another attempt at overcoming the computational challenges associated with attention. We look forward to collaborative progress with the broader community toward breakthroughs in this area.

2.2 Cost-Effectiveness of MoE Models

For sparse computing, we have developed DeepSeekMoE, an advanced **Mixture of Experts (MoE)** architecture, which is illustrated in the lower right part of Figure 1. The advantages of MoE models lie in two folds.

2.2.1 Reducing Computational Requirements for Training. The primary advantage of the MoE architecture lies in its ability to significantly reduce training costs. By selectively activating only a subset of expert parameters, MoE models allow the total parameter count to scale up dramatically while keeping computational requirements modest. For example, **DeepSeek-V2** features 236B parameters, but only 21B parameters are activated per token. Similarly, **DeepSeek-V3** expands to 671B parameters—nearly three

Table 2: Comparison of computational costs for training MoE and dense models: Computational cost per token is measured, assuming a sequence length of 4096.

Model	Size	Training Cost
DeepSeek-V2 MoE	236B	155 GFLOPS/Token
DeepSeek-V3 MoE	671B	250 GFLOPS/Token
Qwen-72B Dense	72B	394 GFLOPS/Token
LLaMa-405B Dense	405B	2448 GFLOPS/Token

times the size of V2—while keeping the activation per token at just 37B. In comparison, dense models such as Qwen2.5-72B and LLaMa3.1-405B require all parameters to be active during training.

As shown in Table 2, the total computational cost for DeepSeek-V3 is approximately 250 GFLOPS per token, whereas the 72B dense model requires 394 GFLOPS and the 405B dense model requires 2448 GFLOPS. This demonstrates that MoE models achieve comparable or even superior performance to dense models while consuming an order of magnitude less computational resources.

2.2.2 Advantages for Personal Use and On-Premises Deployment. In a future where personalized LLM agents [53] become ubiquitous, MoE models offer unique advantages in single-request scenarios. Because only a subset of parameters is activated per request, memory and computational demands are greatly reduced. For example, **DeepSeek-V2** (236B parameters) activates just 21B parameters during inference. This enables PCs with AI SoC chips [6, 10, 58] to achieve nearly 20 tokens per second (TPS), or even twice that speed, which is more than sufficient for personal use. In contrast, dense models of similar capability (e.g., 70B parameters) typically reach only single-digit TPS on similar hardware.

Notably, the increasingly popular KTransformers [39] inference engine allows the complete DeepSeek-V3 model to run on a low-cost server equipped with a consumer GPU (costing approximately \$10,000), while still achieving nearly 20 TPS.

This efficiency makes MoE architectures suitable for local deployments and single-user scenarios, where hardware resources are often limited. By minimizing memory and computational overhead, MoE models can deliver high-quality inference performance without requiring expensive infrastructure.

2.3 Increasing Inference Speed

2.3.1 Overlapping Computation and Communication: Maximizing Throughput. Inference speed encompasses both system-wide maximum throughput and single-request latency. To maximize throughput, our model is architected from the outset to leverage dual micro-batch overlap [31, 78], intentionally overlapping communication latency with computation. As demonstrated in our online inference system and supported by open-source profiling data [31], we decouple the computation of MLA and MoE into two distinct stages. While one micro-batch executes a portion of MLA or MoE computation, the other micro-batch simultaneously performs the corresponding dispatch communication. Conversely, during the computation phase of the second micro-batch, the first micro-batch undergoes the combine communication step. This pipelined approach enables seamless overlap of all-to-all communication with ongoing computation, ensuring that the GPU remains fully utilized at all times. Moreover, in production, we adopt a prefill and decode

disaggregation architecture [80], assigning large batch size prefill and latency-sensitive decode requests to different expert parallelism group sizes. This strategy ultimately maximizes system throughput under real-world service conditions.

2.3.2 Inference Speed Limits. This section focuses on the decode output speed of LLM services, typically measured in **Time Per Output Token (TPOT)**. TPOT is a critical metric for user experience, and it also directly impacts the responsiveness of reasoning models such as OpenAI's o1/o3 and DeepSeek-R1, which rely on the inference length to enhance their intelligence.

For MoE models, achieving high inference speed relies on efficiently deploying expert parameters across computing devices. To achieve the fastest possible inference speed, each device should ideally perform computations for a single expert (or multiple devices should collaboratively compute a single expert if necessary). However, **Expert Parallelism (EP)** requires routing tokens to the appropriate devices, which involves all-to-all communication across the network. As a result, the upper limit of MoE inference speed is dictated by interconnection bandwidth.

Consider a system where each device holds one expert's parameters and processes approximately 32 tokens at a time. This token count strikes a balance between compute-to-memory ratio and communication latency. And this token count ensures that each device processes an equal batch size during expert parallelism, allowing the communication time to be easily calculated.

For a system interconnected with CX7 400Gbps InfiniBand (IB) NICs, the time required for the two all-to-all communications in EP is calculated as follows:

$$\text{Comm. Time} = (1\text{Byte} + 2\text{Bytes}) \times 32 \times 9 \times 7\text{K} / 50\text{GB/s} = 120.96\mu\text{s}$$

Here, dispatch uses FP8 (1 byte), while combine uses BF16 (2 bytes), and the hidden size of each token is approximately 7K. The factor 9 indicates that each token is transferred to 8 routed experts and 1 shared expert. Network latency is not included in this calculation.

As discussed in Section 2.3.1, maximizing throughput necessitates the use of dual micro-batch overlap. In this strategy, our theoretical best-case analysis assumes that computation overhead is minimized, so the upper bound on performance is determined by communication latency. In practical inference workloads, however, request contexts are often much longer, and MLA computations typically dominate execution time. Thus, this analysis represents an idealized scenario under dual micro-batch overlap. Under this assumption, the total time per layer can be formulated as:

$$\text{Total Time Per Layer} = 2 \times 120.96\mu\text{s} = 241.92\mu\text{s}$$

With 61 layers in DeepSeek-V3, the total inference time is:

$$\text{Total Inference Time} = 61 \times 241.92\mu\text{s} = 14.76\text{ms}$$

Thus, the theoretical upper limit for this system is approximately **14.76 ms TPOT**, equivalent to **67 tokens per second**. However, in practice, factors such as communication overhead, latency, incomplete bandwidth utilization, and computational inefficiencies reduce this number.

By contrast, if a high-bandwidth interconnect like GB200 NVL72 (900GB/s unidirectional bandwidth across 72 GPUs) were used, the communication time per EP step drops to:

$$\text{Comm. Time} = (1\text{Byte} + 2\text{Bytes}) \times 32 \times 9 \times 7\text{K} / 900\text{GB/s} = 6.72\mu\text{s}$$

Assuming perfect overlap between computation and communication, this would yield a theoretical upper limit of **0.82 ms TPOT**, or approximately **1200 tokens per second**. However, this figure is purely theoretical and does not account for the substantial drop in GPU efficiency at small batch sizes; in real deployments, actual throughput will be significantly lower. Nonetheless, this calculation vividly illustrates the transformative potential of high-bandwidth scale-up networks in accelerating large-scale model inference.

While MoE models exhibit good scalability, achieving high inference speeds by increasing hardware resources alone is cost-prohibitive. Therefore, software and algorithms must also contribute to improving inference efficiency.

2.3.3 Multi-Token Prediction. Inspired by Gloeckle et al. [36], DeepSeek-V3 introduces a **Multi-Token Prediction (MTP)** framework, which simultaneously enhances model performance and improves inference speed. During inference, traditional autoregressive models generate one token at a decoding step, leading to sequential bottlenecks. MTP mitigates this issue by enabling the model to generate additional candidate tokens at a lower cost and verify them in parallel, similar to previous self-drafting-based speculative decoding approaches [14, 48]. This framework significantly accelerates inference without compromising accuracy.

As illustrated in the top part of Figure 1, each MTP module uses a single layer, which is much more lightweight than the full model, to predict additional tokens, enabling parallel verification of multiple candidate tokens. Although slightly hurting the throughput, this approach significantly improves the end-to-end generation latency. The real world practice data demonstrates that an MTP module achieves an acceptance rate of 80% to 90% for predicting the second subsequent token, which increases the generation TPS by 1.8x compared to the scenario without the MTP module.

Moreover, by predicting multiple tokens per step, MTP increases the inference batch size, which is crucial for boosting EP computational intensity and hardware utilization. Such algorithmic innovations are vital for fast and cost-effective inference in DeepSeek-V3.

2.3.4 High Inference Speed for Reasoning Models and Test-Time Scaling. Test-time scaling in LLMs, exemplified by OpenAI's o1/o3 series [60, 61], has enabled significant advances in mathematical reasoning, programming, and general reasoning by dynamically adjusting computational resources during inference. Subsequent models—including DeepSeek-R1 [28], Claude-3.7 Sonnet [9], Gemini 2.5 Pro [38], Seed1.5-Thinking [68], and Qwen3 [71]—have adopted similar strategies and achieved notable improvements in these tasks.

For these reasoning models, high token output speed is of paramount importance. In reinforcement learning (RL) workflows—such as PPO [67], DPO [64] and GRPO [69]—the necessity to rapidly generate large numbers of samples makes inference throughput a critical bottleneck. Likewise, prolonged reasoning sequences can increase user wait times, reducing the practical usability of such models. As a result, optimizing inference speed through synergistic hardware and software innovations is indispensable for advancing the efficiency of reasoning models. However, effective strategies for accelerating inference and expediting RL training remain active areas of investigation, as discussed in Section 2.1.3. We encourage the broader community to collaboratively explore and develop novel solutions to these ongoing challenges.

2.4 Technique Validation Methodology

Each acceleration technique undergoes rigorous empirical validation to evaluate its accuracy impact, including MLA, FP8 mixed-precision computation, and network co-designed MoE gate routing. Given the prohibitive cost of exhaustive ablation on full-scale models, we adopt a hierarchical and resource-efficient validation pipeline. Each technique is first validated extensively on small-scale models, followed by minimal large-scale tuning, and finally integrated in a single, comprehensive training run. For instance, we first conducted fine-grained FP8 training ablation studies on both 16B and 230B DeepSeek-V2 models before final integration. Under these controlled settings, the relative accuracy loss compared to BF16 remains below 0.25%, attributable to our use of high-precision accumulation and fine-grained quantization strategies.

3 Low-Precision Driven Design

3.1 FP8 Mix-Precision Training

Quantization techniques such as GPTQ [32] and AWQ [51] have been widely used to reduce bit-widths to 8-bit, 4-bit, or even lower, significantly reducing memory requirements. However, these techniques are primarily applied during inference to save memory, rather than in the training phase. NVIDIA’s Transformer Engine has supported FP8 mixed-precision training for some time, but prior to DeepSeek-V3, there were no open-source large models leveraging FP8 for training. Through deep collaboration between our infrastructure and algorithm teams, and after extensive experimentation and innovation, we developed an FP8-compatible training framework for MoE models. Figure 1 shows the computational components where FP8-precision forward and backward processes are utilized in the training pipeline. Fine-grained quantization is applied, i.e., tile-wise 1x128 quantization for activations and block-wise 128x128 quantization for model weights. Further technical details of our FP8 framework are documented in the DeepSeek-V3 technical report [26], and our fine-grained FP8 GEMM implementation has been open-sourced in DeepGEMM [77].

3.1.1 Limitations: While FP8 has great potential for accelerating training, several hardware limitations need to be addressed to fully exploit its capabilities:

- **FP8 Accumulation Precision:** FP8 uses constrained accumulation precision in Tensor Cores, affecting the stability for training large models, particularly on NVIDIA Hopper GPUs. After aligning 32 mantissa products by right-shifting based on the maximum exponent, the Tensor Core only maintains their highest 13 fraction bits for addition, and truncates bits exceeding this range. Addition results are accumulated to FP22 registers (1 sign bit, 8 exponent bits, and 13 mantissa bits).
- **Fine-Grained Quantization Challenges:** Fine-grained quantization such as tile-wise and block-wise quantization introduces large dequantization overhead in transporting the partial results from Tensor Cores to CUDA Cores for scaling factor multiplication. This incurs frequent data movements, reducing computational efficiency and complicating hardware utilization.

3.1.2 Suggestions: To address the limitations of existing hardware, we have the following suggestions for future designs:

- **Increased Accumulation Precision:** Hardware should improve the accumulation register precision to an appropriate value (e.g. FP32), or support a configurable accumulation precision, enabling a trade-off between performance and accuracy for different requirements of training and inference in various models.
- **Native Support for Fine-Grained Quantization:** Hardware should natively support fine-grained quantization, enabling Tensor Cores to receive scaling factors and implement matrix multiplication with group scaling. In this way, the whole partial sum accumulation and dequantization can be completed directly inside Tensor Cores until the final result is produced, avoiding frequent data movements to reduce dequantization overhead. A notable industrial implementation of this approach is NVIDIA Blackwell’s support for **microscaling data format** [66], which exemplifies the practical benefits of native quantization at scale.

3.2 LogFMT: Communication Compression

In the current DeepSeek-V3 architecture, we employ low-precision compression for network communication. During EP parallelism, tokens are dispatched using fine-grained FP8 quantization, reducing communication volume by 50% compared to BF16. This significantly lowers communication time. While the combine stage still uses higher precision (e.g., BF16) due to accuracy requirements, we are actively testing FP8, custom precision formats (e.g., E5M6) and mixing FP8-BF16 for further reductions.

Besides these traditional floating point formats, we also tried a new data type, named **Logarithmic Floating-Point Formats (LogFMT-nBit)**, where n is the number of bits with the leading 1 bit as the sign bit S . By mapping the activations from the original Linear space to the Log space, the distribution of the activations is more uniform. To be specific, given a tile of elements, $[x_1, \dots, x_m]$, which is 1x128 in our implementation, we take the absolute values and compute the logarithm of all the elements, and find the minimum $\min = \log(\text{abs}(x_i))$ and maximum $\max = \log(\text{abs}(x_j))$. The minimum is encoded as $S.00 \dots 01$ and the maximum is encoded as $S.11 \dots 11$, with an interval representing $\text{Step} = \frac{\max - \min}{2^{n-1} - 2}$. Zero values are represented by $S.00 \dots 00$, specially. The left values are rounded to the nearest integer K multiples of Step . The decoding process is simple by combining the sign bit and $\exp^{\min + \text{Step} \times (K-1)}$.

By locally calculating the $\min$ and Step , this data type supports dynamic representation range for different blocks, covering larger ranges or providing more precision, compared to static floating point formats. Besides, we find it is important to round in the original Linear space, instead of the Log space, for the unbiased activation quantization. We also constrain the $\min$ to be larger than $\max - \log(2^{32})$, which means that the max representation range is similar to E5, a floating point with 5 exponents. We validate our LogFMT-nBit on dense language models with around 7 billion parameters, by quantifying the output of the residual branch to simulate the combine stage in MoE models. When setting $n = 8$, sharing the same bits with FP8, the LogFMT-8Bit shows superior training accuracy compared to E4M3 or E5M2. After increasing the n to 10 bits, we find it’s similar to the BF16 combine stage.

3.2.1 Limitations: The initial purpose of using LogFMT is to apply it to activations during transmission or near activation functions, as it offers higher precision than FP8 with the same bit width.

However, subsequent computations require reconversion to BF16 or FP8 to accommodate the Hopper GPU tensor cores' data type. Due to insufficient GPU bandwidth for log/exp operations and excessive register pressure during encode/decode, if encode/decode operations are fused with all-to-all communication, the overhead can be substantial (50%~100%). Therefore, although experimental results validate the effectiveness of this format, we do not employ it eventually.

3.2.2 Suggestions: Providing native support for compression and decompression units tailored to FP8 or custom precision formats represents a viable approach for future hardware. This could help minimize bandwidth requirements and streamline communication pipelines. The reduced communication overhead is particularly helpful in bandwidth-intensive tasks like MoE training.

4 Interconnection Driven Design

4.1 Current Hardware Architecture

The NVIDIA H800 GPU SXM architecture we currently use, illustrated in Figure 2, is built on the Hopper architecture, similar to the H100 GPU. However, it features reduced FP64 computational performance and NVLink bandwidth for regulatory compliance. Specifically, the NVLink bandwidth in H800 SXM nodes is reduced from 900 GB/s to 400 GB/s. This significant reduction in intra-node scale-up bandwidth presents a challenge for high-performance workloads. To compensate, each node is equipped with eight 400G InfiniBand (IB) CX7 NICs, enhancing scale-out capabilities to mitigate the bandwidth deficit.

To address these hardware constraints, the DeepSeek-V3 model incorporates several design considerations that align with the hardware's strengths and limitations.

4.2 Hardware-Aware Parallelism

To align with the constraints of the H800 architecture, the following parallelism strategies were considered to optimize the performance of DeepSeek-V3:

- **Avoidance of Tensor Parallelism (TP):** Tensor Parallelism is avoided during training due to its inefficiency under limited NVLink bandwidth. However, during inference, TP can still be selectively used to improve TTFT and TPOT performance.
- **Enhanced Pipeline Parallelism (PP):** DualPipe [29] is employed to overlap attention and MoE computation with MoE communication. This also reduces pipeline bubbles and balances memory usage across GPUs, improving overall throughput. Additional details are available in the technical report [26].
- **Accelerated Expert Parallelism (EP):** With eight 400Gbps InfiniBand (IB) NICs, the system achieves all-to-all communication at speeds exceeding 40GB/s. Notably, our all-to-all EP implementation, DeepEP [78], is open-sourced, enabling highly efficient expert parallelism as discussed in the following subsection.

4.3 Model Co-Design: Node-Limited Routing

The bandwidth disparity between scale-up (intra-node) and scale-out (inter-node) communication in the H800 architecture is approximately 4:1. Specifically, NVLink provides 200GB/s bandwidth (of which about 160GB/s can actually be achieved), while each 400Gbps

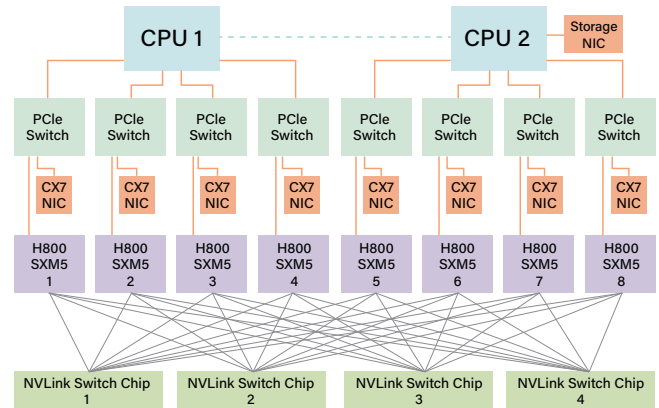


Figure 2: H800 node interconnection.

IB NIC delivers only 50GB/s bandwidth (we consider small message size and latency influence, use 40GB/s for effective bandwidth). To balance and fully utilize the higher intra-node bandwidth, the model architecture is co-designed with hardware, particularly in the **TopK Expert Selection Strategy**.

Consider a setup with 8 nodes (64 GPUs in total) and 256 routed experts (4 experts per GPU). For DeepSeek-V3, each token is routed to one shared expert and 8 routed experts. If its 8 target experts are distributed across all 8 nodes, the communication time over IB would be $8t$, where t represents the time to send one token over IB. However, by leveraging the higher NVLink bandwidth, tokens routed to the same node can be sent once over IB and then forwarded via NVLink to other intra-node GPUs. The NVLink forwarding enables deduplication of the IB traffic. When the target experts for a given token are distributed across M nodes, the deduplicated IB communication cost will be reduced to Mt ($M < 8$).

Since the IB traffic depends on only M , DeepSeek-V3 introduces a **Node-Limited Routing** for the TopK expert selection strategy. Specifically, we group 256 routed experts into 8 groups, with 32 experts per group, and deploy each group on a single node. On top of this deployment, we algorithmically ensure that each token will be routed to up to 4 nodes. This approach mitigates the bottleneck of IB communication and enhances the effective communication bandwidth during training.

4.4 Scale-Up and Scale-Out Convergence

4.4.1 Limitations of Current Implementations. While the Node-Limited Routing strategy reduces communication bandwidth requirements, it complicates communication pipeline kernel implementations due to the disparity in bandwidth between intra-node (NVLink) and inter-node (IB) interconnects. In practice, GPU Streaming Multiprocessors (SM) threads are used for both network message handling (e.g., filling QPs and WQEs) and data forwarding over NVLink, consuming computational resources. For example, during training, up to 20 of the SMs on the H800 GPU are allocated for communication-related operations, leaving fewer resources available for actual computation. To maximize throughput in online inference, we perform EP all-to-all communication entirely through

NIC RDMA, avoiding SM resource contention and improving compute efficiency. This highlights the advantage of RDMA's asynchronous communication model in overlapping computation and communication.

The following are key tasks currently performed by SMs during EP communication, particularly for the combine stage's reduce operations and data type conversions. Offloading these tasks to dedicated communication hardware could free up SMs for computation kernels, significantly improving overall efficiency:

- **Forwarding Data:** Aggregating IB traffic destined for multiple GPUs within the same node between the IB and NVLink domains.
- **Data Transport:** Moving data between RDMA buffers (registered GPU memory regions) and input/output buffers.
- **Reduce Operations:** Executing reduce operations required for EP all-to-all combine communications.
- **Managing Memory Layouts:** Handling fine-grained memory layouts for chunked data transfers across the IB and NVLink domains.
- **Data Type Cast:** Converting data type before and after all-to-all communications.

4.4.2 Suggestions: To address these inefficiencies, we strongly recommend that future hardware should integrate intra-node (scale-up) and inter-node (scale-out) communication into a unified framework. By incorporating dedicated co-processors for network traffic management and seamless forwarding between NVLink and IB domains, such designs can reduce software complexity and maximize bandwidth utilization. For example, node-limited routing strategies employed in DeepSeek-V3 can be further optimized with hardware support for dynamic traffic deduplication.

We also recognize emerging interconnect protocols such as the Ultra Ethernet Consortium (UEC) [17, 18], Ultra Accelerator Link (UALink) [16], both of which are poised to drive advancements in scale-up and scale-out communication. More recently, Unified Bus (UB) [49] has introduced a novel approach to scale-up and scale-out convergence. Section 6 further explores several technical innovations proposed by UEC and UALink. However, in this section, our primary focus is on achieving scale-up and scale-out convergence at the programming framework level:

- (1) **Unified Network Adapter:** Design NICs (Network Interface Cards) or I/O Dies that are connected to unified scale-up and scale-out networks. These adapters should also support basic switch functionality, such as forwarding packets from the scale-out network to specific GPUs within the scale-up network. This could be achieved using a single LID (Local Identifier) or IP address with policy-based routing.
- (2) **Dedicated Communication Co-Processor:** Introduce a dedicated co-processor or programmable component—such as an I/O die—for handling network traffic. This component should offload packet processing from GPU SMs, provide hardware-accelerated memory copy for efficient buffer management, and, crucially, accelerate memory load/store operations in a manner similar to TMA (Tensor Memory Accelerator), thereby saturating bandwidth with minimal resource consumption.
- (3) **Flexible Forwarding, Broadcast and Reduce Mechanisms:** Hardware should support flexible forwarding, broadcast operations (for EP dispatch), and reduce operations (for EP combine) across scale-up and scale-out networks—mirroring our current

GPU SM-based implementation. This would not only improve effective bandwidth but also reduce the computational complexity of network-specific operations.

- (4) **Hardware Synchronization Primitives:** Provide fine-grained hardware synchronization instructions to handle memory consistency issues or out-of-order packet arrivals at the hardware level. This would eliminate the need for software-based synchronization mechanisms like RDMA completion events, which introduce extra latency and increase programming complexity. Memory-semantic communication with an acquire/release mechanism is a promising implementation.

By implementing these recommendations, future hardware designs can significantly enhance the efficiency of large-scale distributed AI systems while simplifying software development.

4.5 Bandwidth Contention and Latency

4.5.1 Limitations: Besides, current hardware lacks the flexibility to dynamically allocate bandwidth between different types of traffic on NVLink and PCIe. For example, during inference, transferring KV cache data from CPU memory to GPU can consume tens of GB/s, saturating PCIe bandwidth. If the GPU simultaneously uses IB for EP communication, this contention between KV cache transfers and EP communication can degrade overall performance and cause latency spikes.

4.5.2 Suggestions:

- **Dynamic NVLink/Pcie Traffic Prioritization:** Hardware should support dynamic prioritization of traffic based on its type. For example, traffic related to EP, TP, and KV cache transfers should be assigned different priorities to maximize interconnect efficiency. For PCIe, exposing the traffic class (TC) to user-level programming would suffice.
- **I/O Die Chiplet Integration:** Integrating NICs directly into the I/O die and connecting them to the compute die in the same package, rather than through conventional PCIe, would substantially reduce communication latency and alleviate PCIe bandwidth contention.

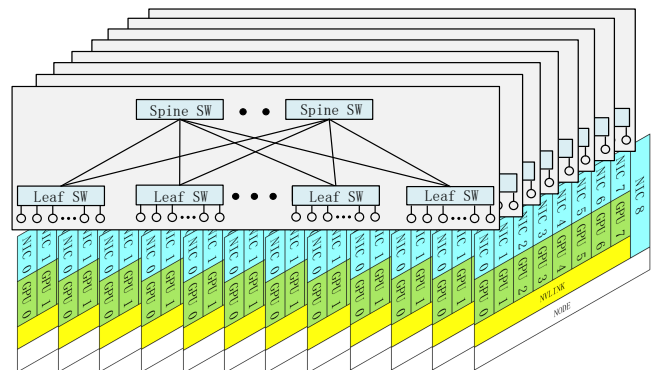


Figure 3: Eight-plane two-layer fat-tree scale-out network: Each GPU and IB NIC pair belongs to one network plane. Cross-plane traffic must use another NIC and PCIe or NVLink for intra-node forwarding.

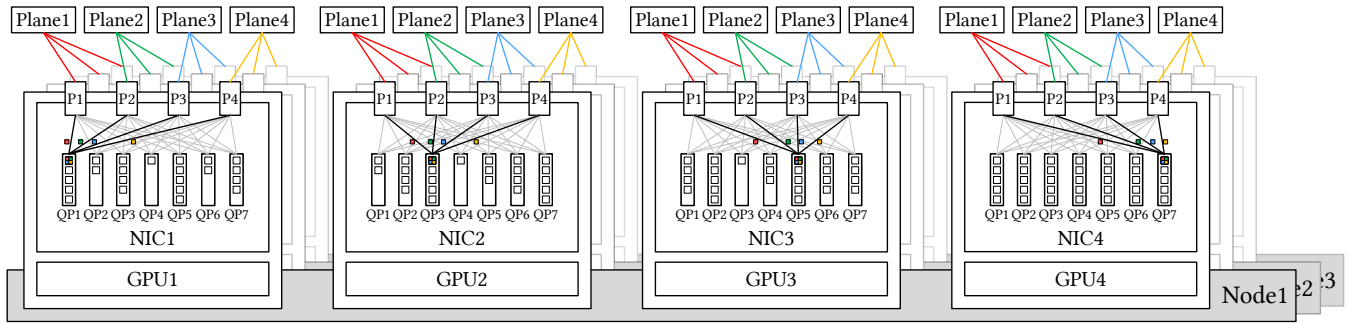


Figure 4: Ideal Multi-Plane Network: Each NIC is equipped with multiple physical ports, each connected to a distinct network plane. A single queue pair (QP) can simultaneously utilize all available ports for transmitting and receiving packets, which necessitates native support for out-of-order placement within the NIC.

- **CPU–GPU Interconnects within the Scale-Up Domain:** To further optimize intra-node communication, CPUs and GPUs should be interconnected using NVLink or similar dedicated high-bandwidth fabrics, rather than relying solely on PCIe. Similar to the benefits provided by integrating NICs into the I/O die, this approach can significantly improve scenarios such as offloading parameters or KV cache between GPU and CPU memory during training and inference.

Table 3: Network topology comparison. Cost estimates are derived from the methodology in the Slim Fly (SF) paper [12]. DF denotes the canonical dragonfly topology [22, 46, 65].

Metric	FT2	MPFT	FT3	SF	DF
Endpoints	2,048	16,384	65,536	32,928	261,632
Switches	96	768	5,120	1,568	16,352
Links	2,048	16,384	131,072	32,928	384,272
Cost [M\$]	9	72	491	146	1,522
Cost/Endpoint [k\$]	4.39	4.39	7.5	4.4	5.8

5 Large Scale Network Driven Design

5.1 Network Co-Design: Multi-Plane Fat-Tree

During the training of DeepSeek-V3, we deployed a **Multi-Plane Fat-Tree (MPFT)** scale-out network, as shown in Figure 3. Each node is equipped with eight GPUs and eight IB NICs, with each GPU–NIC pair assigned to a distinct network plane. Additionally, each node has a 400 Gbps Ethernet RoCE NIC connected to a separate storage network plane for accessing the 3FS [30] distributed file system. In the scale-out network, we used 64-port 400G IB switches, enabling the topology theoretically supports up to 16,384 GPUs while retaining the cost and latency advantages of a two-layer network. However, due to policy and regulatory constraints, just over two thousand GPUs were ultimately deployed.

Furthermore, due to the current limitations of IB ConnectX-7, our deployed MPFT network does not fully realize the envisioned architecture. Ideally, as depicted in Figure 4, each NIC would feature multiple physical ports, each connected to a separate network plane, yet collectively exposed as a single logical interface to the user through port bonding. From a user perspective, a single Queue Pair (QP) could seamlessly transmit and receive messages across all available ports, akin to packet spraying. As a consequence, packets originating from the same QP may traverse distinct network paths and arrive at the receiver out of order, thereby necessitating native support for out-of-order placement within the NIC to guarantee message consistency and preserve the correct ordering semantics. For example, InfiniBand ConnectX-8 natively supports four plane. It would be advantageous for future NICs to fully support advanced multi-plane capabilities, allowing two-tier fat-tree networks to scale

effectively to much larger AI clusters. Overall, the multi-plane architecture offers significant advantages in fault isolation, robustness, load balancing, and large-scale system scalability.

5.1.1 Advantages of Multi-Plane Fat-Tree Network.

- **Subset of Multi-Rail Fat-Tree (MRFT):** The MPFT topology constitutes a specific subset of the broader MRFT architecture. As a result, existing optimizations developed by NVIDIA and NCCL for Multi-Rail networks can be seamlessly leveraged within Multi-Plane network deployments. Furthermore, NCCL’s support for PXN [54] technology addresses the inherent challenge of inter-plane isolation, enabling efficient communication even when direct interconnectivity between planes is absent.
- **Cost Efficiency:** As shown in Table 3, the multi-plane network enables over 10k endpoints using a two-layer fat-tree (FT2) topology, significantly reducing network costs compared to a three-layer fat tree (FT3). The cost per endpoint is even slightly more competitive than the cost-efficient Slim Fly (SF) topology [12].
- **Traffic Isolation:** Each plane operates independently, ensuring that congestion in one plane does not affect others. This isolation improves overall network stability and prevents cascading performance degradation.
- **Latency Reduction:** The two-layer topology achieves lower latency than three-layer fat trees, as demonstrated in our experiments. This makes it particularly suitable for latency-sensitive applications such as MoE-based training and inference.
- **Robustness:** As shown in Figure 4, multi-port NICs provide multiple uplinks, so single-port failures do not disrupt connectivity and rapid, transparent fault recovery is possible.

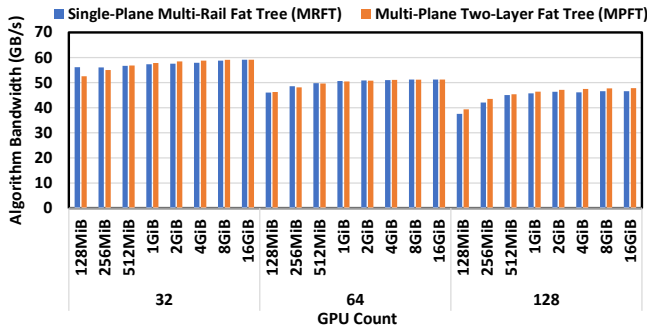


Figure 5: NCCL all-to-all performance from 32 to 128 GPUs for MRFT and MPFT networks.

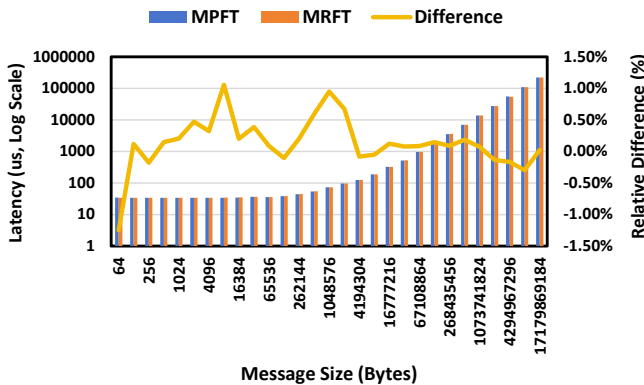


Figure 6: Latency comparison between MPFT and MRFT networks in NCCL all-to-all test under different message sizes, showing that their performance is nearly identical.

It is important to note that, due to current 400G NDR InfiniBand limitations, cross-plane communication requires intra-node forwarding, which introduces additional latency during inference. If future hardware can achieve scale-up and scale-out network convergence as discussed earlier, this latency can be significantly reduced, further enhancing the viability of multi-plane networks.

5.1.2 Performance Analysis. To verify the effectiveness of the Multi-Plane Network design, we conducted real-world experiments on our cluster, modifying the cluster’s network topology to compare the performance of the **Multi-Plane Two-Layer Fat Tree (MPFT)** and the **Single-Plane Multi-Rail Fat Tree (MRFT)**. Below are the key findings from our experiments:

1. All-to-All Communication and EP Scenarios: As illustrated in Figure 5, the all-to-all performance of the multi-plane network is very similar to that of the single-plane multi-rail network. This performance parity can be attributed to NCCL’s PXN [54] mechanism, which optimizes traffic forwarding via NVLink in multi-rail topologies. The multi-plane topology also benefits from this mechanism. As shown in Figure 6, the results of all-to-all communication tests conducted on 16 GPUs reveal negligible differences in latency between the MPFT and MRFT topologies.

To evaluate MPFT’s performance of all-to-all communication in practical training scenarios, we tested the EP communication

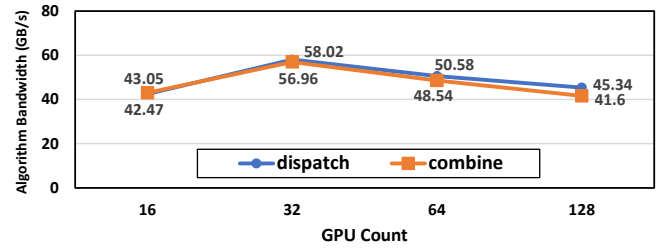


Figure 7: DeepEP performance on MPFT: The EP dispatch and combine kernel communicates across 16 to 128 GPUs using all-to-all. Each GPU processes 4096 tokens. The observed throughput nearly saturates the 400Gps NIC bandwidth.

Table 4: Training metric comparison between MPFT and MRFT networks.

Metric	MPFT	MRFT
tokens/day (B)	272.80	272.52
time/step (s)	19.926	19.946
1F (s)	1.13	1.13
bubble (s)	2.06	2.03
1B (s)	1.99	1.99
1W (s)	0.48	0.48
1F1B (s)	13.95	14.00
opt (s)	0.29	0.31
TFLOPS (non-causal)	432	432
TFLOPS (causal)	385	385
MFU (non-causal)	43.73%	43.68%
MFU (causal)	38.94%	38.90%

patterns commonly used during training. As shown in Figure 7, each GPU achieves a high bandwidth exceeding 40GB/s in a multi-plane network, providing reliable performance that meets the demands of training.

2. Training Throughput for DeepSeek-V3 Model: We also compare the training metrics of the DeepSeek-V3 model between MPFT and MRFT in Table 4. MFU (Model Flops Utilization) is calculated based on BF16 peak performance. Causal MFU only takes into account the flops of the lower triangle of the attention matrix (in line with FlashAttention[19, 20]), while non-causal MFU includes the flops of the whole attention matrix (in line with Megatron [47]). 1F, 1B, and 1W denote forward time, input backward time, and weight backward time, respectively. When training the V3 model on 2048 GPUs, the performance of MPFT is nearly identical to that of MRFT, with observed differences falling within normal fluctuations and measurement error.

5.2 Low Latency Networks

In our model inference, large-scale EP relies heavily on all-to-all communication, which is highly sensitive to both bandwidth and latency. Consider a typical scenario discussed in Section 2.3.2, with a network bandwidth of 50GB/s, the data transfer should ideally take approximately 120 μ s. Therefore, the intrinsic network latencies on the order of microseconds can critically impact system performance, making their effects non-negligible.

Table 5: CPU side end-to-end latency comparison between IB, RoCE, and intra-node NVLink for 64B data transmission.

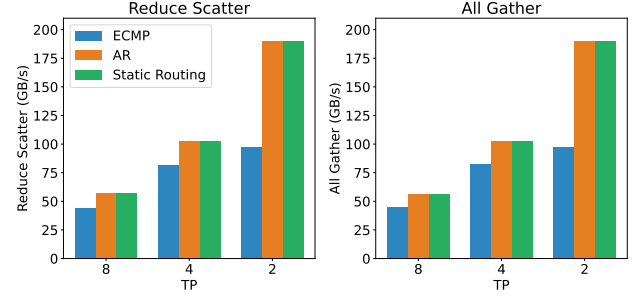
Link Layer	Same Leaf	Cross Leaf
RoCE	3.6us	5.6us
InfiniBand	2.8us	3.7us
NVLink	3.33us	-

5.2.1 IB or RoCE. As shown in Table 5, IB consistently achieves lower latency, making it the preferred choice for latency-sensitive workloads such as distributed training and inference. Although IB has superior latency performance compared to RDMA over Converged Ethernet (RoCE), it comes with certain limitations:

- **Cost:** IB hardware is significantly more expensive than RoCE solutions, which limits its widespread adoption.
- **Scalability:** IB switches typically support only 64 ports per switch, compared to the 128 ports commonly found in RoCE switches. This restricts the scalability of IB-based clusters, particularly for large-scale deployments.

5.2.2 Recommendations for RoCE Improvements. While RoCE has the potential to be a cost-effective alternative to IB, its current limitations in latency and scalability prevent it from fully meeting the demands of large-scale AI systems. Below, we outline specific recommendations for improving RoCE:

- (1) **Specialized Low-Latency RoCE Switches:** We recommend that Ethernet vendors develop RoCE switches specifically optimized for RDMA workloads by removing unnecessary Ethernet features. The Slingshot architecture [22] exemplifies how Ethernet-based designs can achieve latency performance comparable to IB. Similarly, recent innovations from Broadcom [13], including the AI Forwarding Header (AIFH) and upcoming low-latency Ethernet switches, demonstrate the feasibility of high-performance Ethernet fabrics tailored for AI. We are looking forward to continuing innovation in this direction.
- (2) **Optimized Route Policy:** As shown in Figure 8, the default Equal-Cost Multi-Path (ECMP) routing policy in RoCE struggles to distribute traffic efficiently across interconnects, leading to severe congestion performance degradation in NCCL collective communication tests. LLM training traffic, such as in DP (Data Parallelism), tends to lack randomness, causing multiple flows to converge on the same interconnect link. In contrast, Adaptive Routing (AR) [34] can significantly enhance network performance by dynamically spraying packets across multiple paths. While static routing—based on manually configured route tables—can avoid link conflicts for specific destinations, it lacks flexibility. For large-scale all-to-all communication, adaptive routing offers superior performance and scalability.
- (3) **Improved Traffic Isolation or Congestion Control Mechanisms:** Current RoCE switches support only a limited number of priority queues, which are insufficient for complex AI workloads involving concurrent communication patterns such as EP’s all-to-all and DP’s all-reduce. In such mixed workloads, all-to-all traffic can cause incast congestion due to bursty many-to-one transfers, potentially degrading overall network performance. To address incast’s influence on other traffic, one approach is to adopt virtual output queuing (VOQ), assigning a dedicated

**Figure 8: RoCE network bandwidth of AllGather and ReduceScatter communication primitives under different routing methods (ECMP, AR, Static Routing) and TP dimensions.**

virtual queue to each QP to isolate traffic flows. Alternatively, more effective congestion control (CC) mechanisms such as RTT-based CC (RTTCC) or user-programmable CC (PCC) can be employed, enabling NIC-switch co-optimization to maintain low latency and high throughput under dynamic traffic conditions.

5.2.3 InfiniBand GPUDirect Async (IBGDA). We utilize IBGDA [2, 57] to reduce latency in network communications. Traditionally, network communication involves the creation of a CPU proxy thread: once the GPU has prepared the data, it must notify the CPU proxy, which then populates the control information for the work request (WR) and signals the NIC via a doorbell mechanism to initiate data transmission. This process introduces additional communication overhead.

IBGDA addresses this issue by allowing the GPU to directly fill the WR content and write to the RDMA doorbell MMIO address. By managing the entire control plane within the GPU, IBGDA eliminates the significant latency overhead associated with GPU-CPU communication. Moreover, when sending a large number of small packets, the control plane processor can easily become a bottleneck. Since GPUs have multiple parallel threads, the sender can leverage these threads to distribute the workload, thereby avoiding such bottlenecks. A range of works—including our DeepEP [78]—have leveraged IBGDA and reported substantial performance gains [1, 15, 79]. We therefore advocate for such capabilities to be widely supported across accelerator devices.

6 Discussion and Insights for Future Hardware Architecture Design

Building on the previous sections, we summarize key architectural insights and outline future directions for hardware design tailored to large-scale AI workloads.

Section 2.3.2 highlighted the importance of large-scale scale-up networks for accelerating model inference. Section 3 discussed the necessity of efficient support for low-precision computation and communication. Section 4 explored the convergence of scale-up and scale-out architectures, along with several proposed enhancements. Section 5 focused on multi-plane network topologies and identified key improvements needed for Ethernet-based interconnects.

Together, these sections identify hardware limitations in concrete application contexts and offer corresponding suggestions. Building on that foundation, this section expands the discussion to broader considerations and proposes forward-looking directions for future hardware architecture design.

6.1 Robustness Challenges

6.1.1 Limitations:

- **Interconnect Failures:** High-performance interconnects (e.g., IB and NVLink) are prone to intermittent disconnections, which can disrupt node-to-node communication. This is especially harmful in communication-heavy workloads like EP, where even brief interruptions may lead to significant performance drops or job failures.
- **Single Hardware Failures:** Node crashes, GPU failures, or ECC (Error-Correcting Code) memory errors can compromise long-running training jobs, often requiring costly restarts. The impact of such failures escalates in large-scale deployments, where the probability of a single-point failure increases proportionally with system size.
- **Silent Data Corruption:** Errors undetected by ECC mechanisms, such as multi-bit memory flips or computational inaccuracies, pose a significant risk to model quality. These errors are particularly insidious in long-running tasks, as they can propagate undetected and corrupt downstream computations. Current mitigation strategies rely on application-level heuristics, which are insufficient for ensuring system-wide robustness.

6.1.2 Suggestions for Advanced Error Detection and Correction. To mitigate risks associated with silent corruption, hardware must incorporate advanced error detection mechanisms beyond traditional ECC. Techniques such as checksum-based validation or hardware-accelerated redundancy checks can provide higher reliability for large-scale deployments.

Furthermore, hardware vendors should deliver comprehensive diagnostic toolkits to end users, empowering them to rigorously verify the integrity of their systems and proactively identify any latent silent data corruption. Such toolkits, when embedded as part of the standard hardware package, foster transparency and enable continuous validation throughout the operational lifecycle, thereby bolstering overall system trustworthiness.

6.2 CPU Bottlenecks and Interconnects

While accelerator design often takes center stage, CPUs remain essential for coordinating computation, managing I/O, and sustaining system throughput. However, current architectures face several critical bottlenecks:

First, as discussed in Section 4.5, the PCIe interface between CPUs and GPUs often becomes a bandwidth bottleneck, particularly during large-scale parameter, gradient, or KV cache transfers. To mitigate this, future systems should adopt direct CPU–GPU interconnects—such as NVLink or Infinity Fabric—or integrate both CPUs and GPUs into the scale-up domain, thereby eliminating intra-node bottlenecks.

In addition to PCIe limitations, sustaining such high data transfer rates also requires exceptionally high memory bandwidth. For example, saturating 160 lanes of PCIe 5.0 demands over 640 GB/s

per node, translating to a memory bandwidth requirement of approximately 1 TB/s per node—posing a significant challenge for conventional DRAM architectures.

Lastly, latency-sensitive tasks such as kernel launches and network processing demand high single-core CPU performance, typically requiring base frequencies above 4 GHz. Furthermore, modern AI workloads require sufficient CPU cores per GPU to prevent control-side bottlenecks. For chiplet-based architectures, additional cores are needed to support cache-aware workload partitioning and isolation.

6.3 Toward Intelligent Networks for AI

To meet the demands of latency-sensitive workloads, future interconnects must prioritize both low latency and intelligent networks:

- **Co-Packaged Optics:** Incorporating silicon photonics enables scalable higher bandwidth scalability and enhanced energy efficiency, both are critical for large-scale distributed systems.
- **Lossless Network:** Credit-Based Flow Control (CBFC) mechanisms ensure lossless data transmission, yet naively triggering flow control can induce severe head-of-line blocking. Therefore, it is imperative to deploy advanced, endpoint-driven congestion control (CC) algorithms that proactively regulate injection rates and avert pathological congestion scenarios.
- **Adaptive Routing:** As underscored in Section 5.2.2, future network should standardize the adoption of dynamic routing schemes—such as packet spraying and congestion-aware path selection—that continuously monitor real-time network conditions and intelligently redistribute traffic. These adaptive strategies are particularly effective in alleviating hotspots and mitigating bottlenecks during collective communication workloads, including all-to-all and reduce-scatter operations.
- **Efficient Fault-Tolerant Protocols:** Robustness against failures can be significantly enhanced through the deployment of self-healing protocols, redundant ports, and rapid failover techniques. For instance, link-layer retry mechanisms and selective retransmission protocols prove indispensable in scaling reliability across large networks, minimizing downtime and ensuring seamless operation despite intermittent failures.
- **Dynamic Resource Management:** To handle mixed workloads effectively, future hardware should enable dynamic bandwidth allocation and traffic prioritization. For example, inference tasks should be isolated from training traffic in unified clusters, ensuring responsiveness for latency-sensitive applications.

6.4 Discussion on Memory-Semantic Communication and Ordering Issue

Inter-node communication using load/store memory semantics is efficient and programmer-friendly, but current implementations are hampered by memory ordering challenges. For example, after writing data, the sender must issue an explicit memory barrier (fence) before updating a flag to notify the receiver, ensuring data consistency. This strict ordering introduces additional round-trip time (RTT) latency and can stall the issuing thread, impeding in-flight stores and reducing throughput. Similar out-of-order synchronization issues arise in message-semantic RDMA; for instance, performing RDMA atomic add operations with packet spraying

after regular RDMA writes on InfiniBand or NVIDIA BlueField-3 can incur additional RTT latency.

To address these, we advocate for hardware support that offers built-in ordering guarantees for memory-semantic communication. Such consistency should be enforced both at the programming level (e.g., via acquire/release semantics) and by hardware at the receiver, enabling in-order delivery without added overhead.

Several approaches are possible. For example, the receiver can buffer atomic messages and use packet sequence numbers (PSN) to ensure in-order processing; this method is straightforward and effective for maintaining correctness. Alternatively, a region-based acquire/release (RAR) mechanism is particularly attractive, as it enables true acquire/release semantics for remote memory access and offers greater flexibility for a range of workloads. In this approach, the receiver hardware maintains lightweight metadata—such as bitmaps or region-based counters—to track the state of memory regions. Acquire and release operations are scoped to specific address ranges, enabling efficient, hardware-enforced ordering without explicit sender-side fences. Notably, both approaches are amenable to implementation on the NIC or I/O die and are applicable to both memory-semantic and message-semantic RDMA primitives, thereby broadening their practical utility.

6.5 In-Network Computation and Compression

EP involves two critical all-to-all stages—**dispatch** and **combine**—that present significant opportunities for in-network optimization. The **dispatch** stage resembles a small-scale multicast operation, where a single message must be forwarded to multiple target devices. A hardware-level protocol enabling automatic packet replication and forwarding to multiple destinations could drastically reduce communication overhead and improve efficiency.

The **combine** stage, acting as a small-scale reduction operation, could benefit from in-network aggregation techniques. However, due to the small reduction scope and imbalanced workload in EP combine, implementing in-network aggregation in a flexible manner is challenging.

Moreover, as highlighted in Section 3.2, LogFMT enables low-precision token transmission with minimal impact on model performance. Incorporating LogFMT natively within network hardware could further optimize communication by increasing entropy density and reducing bandwidth usage. Hardware-accelerated compression and decompression would allow seamless integration of LogFMT into distributed systems, enhancing overall throughput.

6.6 Memory-Centric Innovations

6.6.1 Limitations of Memory Bandwidth. The exponential growth in model sizes has outpaced advancements in high-bandwidth memory (HBM) technology. This disparity creates a memory bottleneck, particularly in attention-heavy architectures like Transformers.

6.6.2 Suggestions:

- **DRAM-Stacked Accelerators:** Leveraging advanced 3D stacking technologies, DRAM dies can be vertically integrated atop a logic die, thereby enabling exceptionally high memory bandwidth, ultra-low latency, and a practical memory capacity (though stack-limited). This architectural paradigm proves remarkably advantageous for ultra-fast inference in MoE models, where

memory throughput is a critical bottleneck. Architectures such as SeDRAM[72] exemplify the potential of this approach, delivering unprecedented performance for memory-bound workloads.

- **System-on-Wafer (SoW):** Wafer-scale integration [50] can maximize computational density and memory bandwidth, addressing the needs of ultra-large-scale models.

7 Conclusion

DeepSeek-V3 exemplifies the transformative potential of hardware-software co-design in advancing the scalability, efficiency, and robustness of large-scale AI systems. By addressing the limitations of current hardware architectures and proposing actionable recommendations, this paper provides a roadmap for the next generation of AI-optimized hardware. These innovations will be critical as AI workloads continue to grow in complexity and scale, driving the future of intelligent systems.

References

- [1] Elena Agostini, Davide Rossetti, and Sreeram Potluri. 2017. Offloading Communication Control Logic in GPU Accelerated Applications. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. 248–257. <https://doi.org/10.1109/CCGRID.2017.29>
- [2] E. Agostini, D. Rossetti, and S. Potluri. 2018. GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters. *J. Parallel and Distrib. Comput.* 114 (2018), 28–45. <https://doi.org/10.1016/j.jpdc.2017.12.007>
- [3] AI@Meta. 2024. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [4] AI@Meta. 2024. Llama 3.1 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md
- [5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [6] AMD. 2025. AMD Ryzen AI Max+ PRO 395: Designed to power a new generation of compact Copilot+ PC workstations. <https://www.amd.com/en/products/processors/laptop/ryzen-pro/ai-max-pro-300-series/amd-ryzen-ai-max-plus-pro-395.html>
- [7] Wei An, Xiao Bi, Guanting Chen, Shanhuang Chen, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Wenjun Gao, Kang Guan, Jianzhong Guo, Yongqiang Guo, Zhe Fu, Ying He, Panpan Huang, Jia Shi Li, Wenfeng Liang, Xiaodong Liu, Xin Liu, Yiyuan Liu, Yuxuan Liu, Shanghao Lu, Xuan Lu, Xiaotao Nie, Tian Pei, Junjie Qiu, Hui Qu, Zehui Ren, Zhangli Sha, Xuecheng Su, Xiaowen Sun, Yixuan Tan, Minghui Tang, Shiyu Wang, Yaohui Wang, Yongji Wang, Ziwei Xie, Yiliang Xiong, Yanhong Xu, Shengfeng Ye, Shuiping Yu, Yukun Zha, Liyue Zhang, Haowei Zhang, Mingchuan Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, and Yuheng Zou. 2024. Fire-Flyer AI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–23. <https://doi.org/10.1109/SC41406.2024.00089>
- [8] Anthropic. 2024. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>
- [9] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>
- [10] Apple. 2024. Apple introduces M4 Pro and M4 Max. <https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/>
- [11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150* (2020).
- [12] Nils Blach, Maciej Besta, Daniele De Sensi, Jens Domke, Hussein Harake, Shigang Li, Patrick Iff, Marek Konieczny, Kartik Lakhotia, Ales Kubicek, Marcel Ferrari, Fabrizio Petrini, and Torsten Hoeftler. 2025. A high-performance design, implementation, deployment, and evaluation of the slim fly network. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI'24). USENIX Association, USA, Article 57, 20 pages.
- [13] Broadcom. 2025. Scale Up Ethernet Framework. <https://docs.broadcom.com/doc/scale-up-ethernet-framework>
- [14] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net. <https://openreview.net/forum?id=PEpbUobfjv>

- [15] Shaoyuan Chen, Wencong Xiao, Yutong Lin, Mingxing Zhang, Yingdi Shan, Jinlei Jiang, Kang Chen, and Yongwei Wu. 2025. Efficient Heterogeneous Large Language Model Decoding with Model-Attention Disaggregation. *arXiv:2405.01814* [cs.LG] <https://arxiv.org/abs/2405.01814>
- [16] ULTRA ACCELERATOR LINK CONSORTIUM. 2025. Introducing UALink 200G 1.0 Specification. https://ualinkconsortium.org/wp-content/uploads/2025/04/UALink-1.0-White_Paper_FINAL.pdf
- [17] Ultra Ethernet Consortium. 2023. Overview of and Motivation for the Forthcoming Ultra Ethernet Consortium Specification. <https://ultraethernet.org/wp-content/uploads/sites/20/2023/10/23.07.12-UEC-1.0-Overview-FINAL-WITH-LOGO.pdf>
- [18] Ultra Ethernet Consortium. 2024. UEC Progresses Towards v1.0 Set of Specifications. <https://ultraethernet.org/uec-progresses-towards-v1.0-set-of-specifications/>
- [19] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.
- [20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- [21] Tri Dao and Albert Gu. 2024. Transformers are SSMs: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 399, 31 pages.
- [22] Daniele De Sensi, Salvatore Di Girolamo, Kim H. McMahon, Duncan Roweth, and Torsten Hoeftler. 2020. An In-Depth Analysis of the Slingshot Interconnect. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. <https://doi.org/10.1109/SC41405.2020.00039>
- [23] DeepSeek-AI. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *CoRR* abs/2406.11931 (2024). <https://doi.org/10.48550/arXiv.2406.11931>
- [24] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *CoRR* abs/2401.02954 (2024). <https://doi.org/10.48550/arXiv.2401.02954>
- [25] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *CoRR* abs/2405.04434 (2024). <https://doi.org/10.48550/arXiv.2405.04434>
- [26] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. (2024). *arXiv:2412.19437* [cs.CL] <https://arxiv.org/abs/2412.19437>
- [27] DeepSeek-AI. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *CoRR* abs/2401.06066 (2024). <https://doi.org/10.48550/arXiv.2401.06066>
- [28] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL] <https://arxiv.org/abs/2501.12948>
- [29] DeepSeek-AI. 2025. DualPipe: A bidirectional pipeline parallelism algorithm for computation-communication overlap in V3/R1 training. <https://github.com/deepseek-ai/dualpipe>.
- [30] DeepSeek-AI. 2025. Fire-Flyer File System. <https://github.com/deepseek-ai/3FS>
- [31] DeepSeek-AI. 2025. Profiling Data in DeepSeek Infra. <https://github.com/deepseek-ai/profile-data?tab=readme-ov-file#inference>
- [32] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [33] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 57–70. <https://doi.org/10.1145/3651890.3672233>
- [34] Patrick Geoffroy and Torsten Hoeftler. 2008. Adaptive Routing Strategies for Modern High Performance Networks. In *2008 16th IEEE Symposium on High Performance Interconnects*. 165–172. <https://doi.org/10.1109/HOTI.2008.21>
- [35] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 03 (May 2024), 33–39. <https://doi.org/10.1109/MM.2024.3373763>
- [36] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & Faster Large Language Models via Multi-token Prediction. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net. <https://openreview.net/forum?id=pEWAcjU2>
- [37] Google. 2024. Introducing Gemini 2.0: our new AI model for the agentic era. <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/>
- [38] Google. 2025. Gemini 2.5: Our most intelligent AI model. <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>
- [39] MADSys group and Approaching.AI. 2025. A Flexible Framework for Experiencing Cutting-edge LLM Inference Optimizations. <https://github.com/kvcache-ai/ktransformers>
- [40] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079* (2024).
- [41] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [42] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. <https://arxiv.org/abs/2402.15627> *arXiv:2402.15627* [cs].
- [43] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. <https://doi.org/10.1145/3579371.3589350>
- [44] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. 2024. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. *arXiv:2403.05527* [cs.LG]
- [45] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *CoRR* abs/2001.08361 (2020). *arXiv:2001.08361* <https://arxiv.org/abs/2001.08361>
- [46] John Kim, William J. Dally, Steve Scott, and Dennis Abts. 2008. Technology-Driven, Highly-Scalable Dragonfly Topology. In *2008 International Symposium on Computer Architecture*. 77–88. <https://doi.org/10.1109/ISCA.2008.19>
- [47] Vijay Anand Korthikanti, Jared Casper, Sangkuk Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023).
- [48] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net. <https://openreview.net/forum?id=1NdN7eXyb4>
- [49] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, Jianbing Wang, Xiangyu Chen, Peng Dong, Rui Meng, Wenjie Liu, Zhe Zhou, Ziyang Zhang, Yuhang Gai, Cunle Qian, Yi Xiong, Zhongwu Cheng, Jing Xia, Yuli Ma, Xi Chen, Wenhua Du, Shizhong Xiao, Chungang Li, Yong Qin, Liudong Xiong, Zhou Yu, Lv Chen, Lei Chen, Buyun Wang, Pei Wu, Junen Gao, Xiaochu Li, Jian He, Shizhuan Yan, and Bill McColl. 2025. UB-Mesh: A Hierarchically Localized nD-FullMesh Datacenter Network Architecture. *arXiv:2503.20377* [cs.AR] <https://arxiv.org/abs/2503.20377>
- [50] Sean Lie. 2022. Cerebras Architecture Deep Dive: First Look Inside the HW/SW Co-Design for Deep Learning : Cerebras Systems. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–34. <https://doi.org/10.1109/HCS55958.2022.9895479>
- [51] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*.
- [52] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv preprint arXiv:2402.02750* (2024).
- [53] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. *arXiv preprint arXiv:2503.21460* (2025).
- [54] Karthik Mandakolathur and Sylvain Jaeger. 2022. Doubling all2all Performance with NVIDIA Collective Communication Library 2.12. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>
- [55] Mistral. 2024. Cheaper, Better, Faster, Stronger: Continuing to push the frontier of AI and making it accessible to all. <https://mistral.ai/news/mixtral-8x22b>
- [56] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie Amy Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie

- Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, K. R. Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. 2023. Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models. <https://arxiv.org/abs/2104.05158> [cs].
- [57] NVIDIA. 2022. Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/>
- [58] NVIDIA. 2025. NVIDIA DGX Spark: A Grace Blackwell AI supercomputer on your desk. <https://www.nvidia.com/en-us/products/workstations/dgx-spark/>
- [59] OpenAI. 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [60] OpenAI. 2024. Introducing OpenAI o1. <https://openai.com/o1/>
- [61] OpenAI. 2025. Introducing OpenAI o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>
- [62] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 691–706. <https://doi.org/10.1145/3651890.3672265>
- [63] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. 2024. Various lengths, constant speed: efficient language modeling with lightning attention. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML '24). JMLR.org, Article 1688, 19 pages.
- [64] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG] <https://arxiv.org/abs/2305.18290>
- [65] Md Shafayat Rahman, Saptarshi Bhowmik, Yevgeniy Ryasnianskiy, Xin Yuan, and Michael Lang. 2019. Topology-custom UGAL routing on dragonfly. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, Colorado) (SC '19). Association for Computing Machinery, New York, NY, USA, Article 17, 15 pages. <https://doi.org/10.1145/3295500.3356208>
- [66] Bitan Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Venmugil Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmahosroshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Siu, Pradeep Dubey, Paulius Mickevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. 2023. Microscaling Data Formats for Deep Learning. arXiv:2310.10537 [cs.LG] <https://arxiv.org/abs/2310.10537>
- [67] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] <https://arxiv.org/abs/1707.06347>
- [68] ByteDance Seed. 2025. Seed1.5-Thinking: Advancing Superb Reasoning Models with Reinforcement Learning. arXiv:2504.13914 [cs.CL] <https://arxiv.org/abs/2504.13914>
- [69] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>
- [70] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. CoRR abs/1911.02150 (2019). <http://arxiv.org/abs/1911.02150>
- [71] Qwen Team. 2025. Qwen3: Think Deeper, Act Faster. <https://github.com/QwenLM/Qwen3>
- [72] Song Wang, Bing Yu, Wenwu Xiao, Fujun Bai, Xiaodong Long, Liang Bai, Xuerong Jia, Fengguo Zuo, Jie Tan, Yixin Guo, Peng Sun, Jun Zhou, Qiong Zhan, Sheng Hu, Yu Zhou, Yi Kang, Qiwei Ren, and Xiping Jiang. 2023. A 135 GBps/Gbit 0.66 pJ/bit Stacked Embedded DRAM with Multilayer Arrays by Fine Pitch Hybrid Bonding and Mini-TSV. In *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 1–2. <https://doi.org/10.23919/VLSITechnologyandCirc57934.2023.10185427>
- [73] xAI. 2024. Grok-2 Beta Release. <https://x.ai/news/grok-2>.
- [74] xAI. 2024. Our Gigafactory of Compute:Colossus. <https://x.ai/colossus>.
- [75] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024).
- [76] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. <https://arxiv.org/abs/2502.11089>
- [77] Chenggang Zhao, Liang Zhao, Jiashi Li, and Zhean Xu. 2025. DeepGEMM: clean and efficient FP8 GEMM kernels with fine-grained scaling. <https://github.com/deepseek-ai/DeepGEMM>.
- [78] Chenggang Zhao, Shangyan Zhou, Liye Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>.
- [79] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. arXiv:2503.20313 [cs.DC] <https://arxiv.org/abs/2503.20313>
- [80] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>